

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 1

Basic question 1: Hash tables

Insert the following values into initially empty hash tables, in the specified order:

Axe, Bo, Center, D, E, Five, Graph

Use the following (poor-quality) hash function, with modular compression:

$h(x) = \text{length of } x$ (remember to double-check the long words!)

You do not have to resize the hash tables.

(a) Use a linear probing hash table with capacity 10.

0	
1	D
2	Bo
3	Axe
4	E
5	Five
6	Center
7	Graph
8	
9	

(b) Use a separate chaining hash table with capacity 5, and standard linked lists.

0	→ Graph
1	→ Center → D → E *
2	→ Bo
3	→ Axe
4	→ Five

* or: → E → D → Center

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 2

Basic question 2: Heaps

Here is a binary min-heap of integers in array form, with an unknown value x :

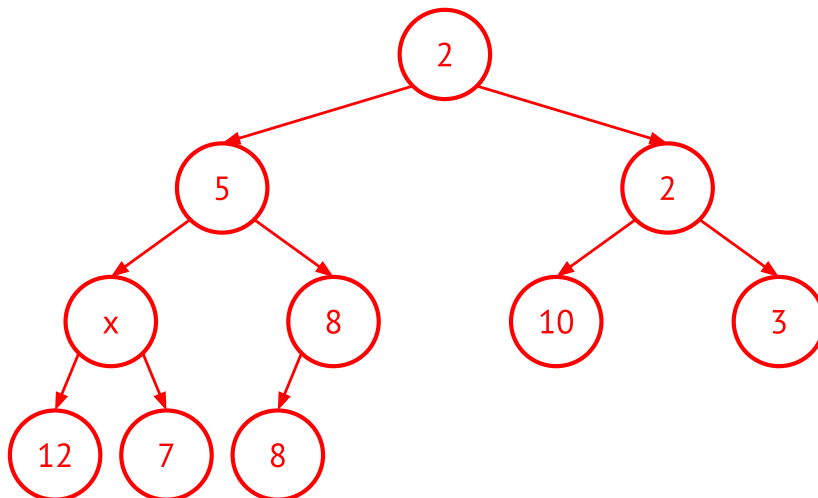
0	1	2	3	4	5	6	7	8	9
2	5	2	x	8	10	3	12	7	8

List all the possible values for x :

5, 6, 7

Explain why this is the case.

Drawing the heap as a tree, we see that x has parent 5 and children 12 and 7:



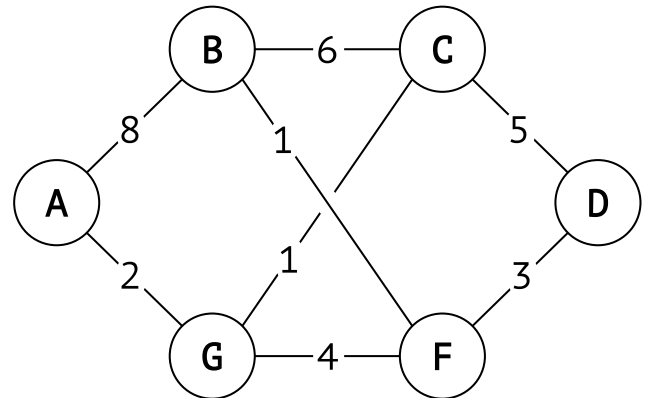
The heap invariant requires $5 \leq x$ and $x \leq 12$ and $x \leq 7$. So $x \in \{5, 6, 7\}$.

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 3

Basic question 3: Graphs

Construct (a) a minimum spanning tree (MST), and (b) a shortest path tree (SPT) rooted at **D**, for the weighted undirected graph on the right.

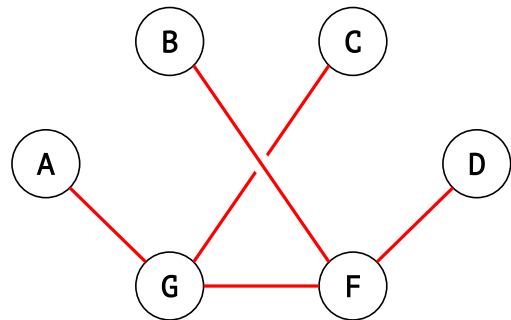
For (a) you should run Prim's algorithm from **A**, and for (b) you should run Dijkstra's from **D**.



(a) *Prim's algorithm starting in A*: How does the MST look? (Draw the edges.)

List the vertices in the order they are visited by Prim's algorithm:

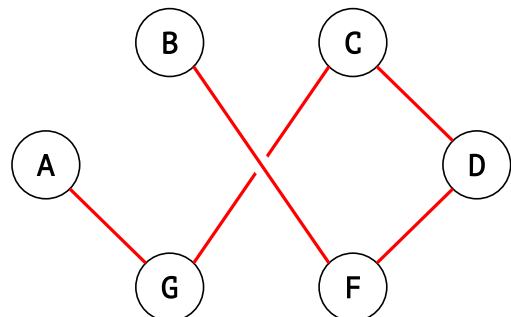
A, G, C, F, B, D



(b) *Dijkstra's algorithm starting in D*: How does the SPT look? (Draw the edges.)

List the vertices in the order they are visited by Dijkstra's algorithm:

D, F, B, C, G, A



CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 4

Basic question 4: Insertion and selection sort

Perform in-place insertion sort *and* in-place selection sort on the array [81, 63, 32, 75, 59, 41].

Show the array after each iteration of the outer loop in the algorithm.

Insertion sort

0	1	2	3	4	5
81	63	32	75	59	41
63	81	32	75	59	41
32	63	81	75	59	41
32	63	75	81	59	41
32	59	63	75	81	41
32	41	59	63	75	81

Selection sort

0	1	2	3	4	5
81	63	32	75	59	41
32	63	81	75	59	41
32	41	81	75	59	63
32	41	59	75	81	63
32	41	59	63	81	75
32	41	59	63	75	81

Getting the names mixed up is fine, as long as one is insertion sort and the other is selection sort.

It's fine to use a version of either algorithm whose outer loop goes over the array from right to left (for example, selecting a maximal remaining element in selection sort).

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 5

Basic question 5: Complexity

The following function takes an array of **distinct** positive integers (that is, no duplicates) and produces a new larger array with the same elements and some cells that are null, much as resizing works in a linear probing hash table.

build_table(arr: integer array) → (integer or null) array:

newLength = arr.length · 2

newArr = new array of length newLength, all cells null

for x **in** arr:

ix = x **mod** newLength

while newArr[ix] **is not** null:

ix = (ix + 1) **mod** newLength

newArr[ix] = x

return newArr

Below, we calculate the total cost each line contributes. Other ways are also possible.

total cost $O(n)$

$O(n)$ iterations

total cost $O(1) \cdot O(n) = O(n)$

$O(n)$ iterations

total cost $O(1) \cdot O(n) \cdot O(n) = O(n^2)$

total cost $O(1) \cdot O(n)$

- a) What is the asymptotic **worst-case** complexity of ^{build_table(arr)}resize(arr) in terms of the length n of arr? Justify your answers with comments in the margin above.

Complexity: $O(n^2)$

- b) Construct a worst-case input of size 5 (an array of length 5) for which the algorithm is as slow as possible. Keep in mind that there can be no duplicate values!

10	20	30	40	50
----	----	----	----	----

Any array where all values collide modulo 10 (another example is [1,11,21,31,41]).

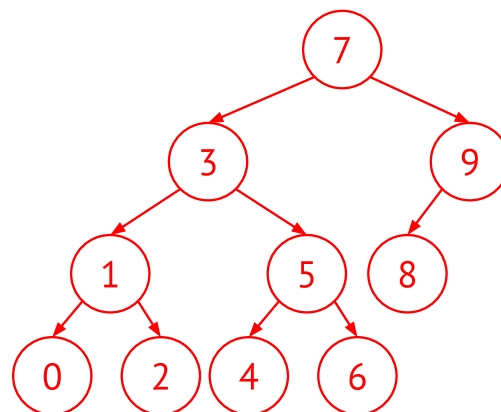
CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 6

Basic question 6: AVL size imbalance

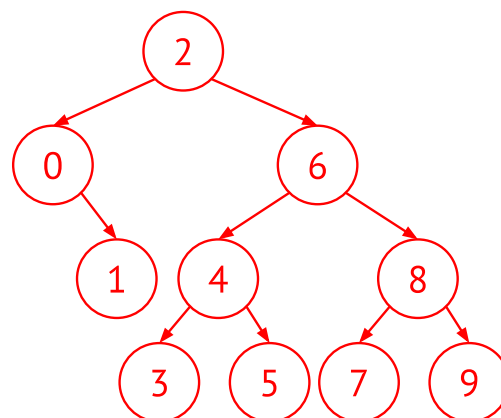
Recall that AVL trees satisfy a height balance invariant: *the heights of siblings differ by at most 1*.

However, the **sizes** of siblings can differ more. Draw an AVL tree with 10 nodes, containing the numbers 0 to 9, such that the sizes of the two children of the root differ as much as possible.

Write L and R for the sizes of the children of the root. Here is an example with $L = 7$ and $R = 2$, for a size difference of 5 (the other such example is obtained by right rotating at $9 \rightarrow 8$):



Alternatively, one can achieve $L = 2$ and $R = 7$ by mirroring the shape:



(Optional proof of optimality.) A binary tree with height at most 3 has size at most $2^3 - 1 = 7$. Since our AVL tree has size 10, it has height at least 4. Therefore, one of the children of the root has height at least 3. By the AVL invariant, the other child has height at least 2. Since the size is always at least the height, both children of the root have size at least 2. Since the sum of their sizes is $10 - 1 = 9$, their size is at most $9 - 2 = 7$. That means their size difference is at most $7 - 2 = 5$.

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 7

Advanced question 7: Remove from a strange tree

A *strange tree* is a data structure storing a set of values. It is a binary tree, but not a BST.

A strange tree is either *null* or has a value and left and right children satisfying the following:

- The value is *less than* all values in its subtrees (that is, a strict version of the heap property holds).
- All the values in the left subtree are *less than* all the values in the right subtree.
- Both children are also strange trees.
- An example strange tree is to the right (*null* pointers are omitted).

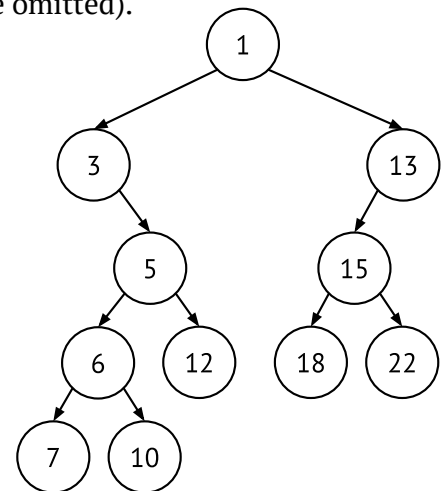
You may assume that the StrangeTree class looks like this:

```
class StrangeTree:
    root: TreeNode

class TreeNode:
    value: int
    left, right: TreeNode
```

Your task is to write two functions that remove the minimal and maximum values from a strange tree, respectively:

- removeMin(tree: StrangeTree)
- removeMax(tree: StrangeTree)



Notes:

- You can assume that the input tree is not *null*.
- The resulting tree must still be a strange tree, of course.
- Your solution should take $O(h)$ time in terms of the height h of the tree.
- You may answer using pseudocode or Java/Python/Haskell.
- You can define helper functions if you want.

```
removeMin(tree):
    tree.root = minHelper(tree.root)
```

```
minHelper(node):
    if node.left is null:
        return node.right
    node.value = node.left.value
    node.left = minHelper(node.left)
    return node
```

```
removeMax(tree):
    tree.root = maxHelper(tree.root)
```

```
maxHelper(node):
    if node.right is not null:
        node.right = maxHelper(node.right)
    else if node.left is not null:
        node.left = maxHelper(node.left)
    else
        node = null
    return node
```

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 8

Advanced question 8: Complexity

You have a database of the N most prominent European rivers. The database consists of a number of sorted arrays of length N , including the following:

- **rivers** stores the names of the rivers, in alphabetical order
- **lengths** stores the length of each river, in *the same order* as the **rivers** array
- **length_index** contains the river names, but sorted by *the length of the river*
- (there can be more arrays storing the basin size, or the average discharge, or other things for each river – but we ignore them here)

Here is an example of how the database can look:

rivers		lengths		length_index	
0	Adige	0	410 km	0	Yantra
1	Belaya	1	1430 km	1	Thames
2	Dalälven	2	520 km	2	Adige
3	Danube	3	2860 km	3	Dalälven
...		...		...	
$N-2$	Volga	$N-2$	3690 km	$N-2$	Danube
$N-1$	Yantra	$N-1$	285 km	$N-1$	Volga

Consider the following problem:

“Given a length K kilometers, which is the longest river that is shorter than K ?”

We want to solve this problem using a fast algorithm. What is the best asymptotic complexity in N you can achieve? Write your answer in O -notation. Justify by briefly sketching your algorithm.

- Treat number and string comparisons as taking constant time.
- You do not have to justify why there is no better algorithm.

Complexity: $O(\log(N)^2)$

Justification (algorithm sketch):

Note first that the arrays **rivers** and **lengths** can serve as an immutable map with river names as keys and their length as values. We use binary search to look up the length for a river ($O(\log(N))$).

To find the longest river that is shorter than K , we perform a binary search in the array **length_index**. Each step uses a comparison of the current river length with K , which we said is $O(\log(N))$. Since there are $O(\log(N))$ many steps in binary search, the total runtime is $O(\log(N)^2)$.

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 9

Advanced question 9: Mystery data structure

Below is pseudo-code for a data structure from the course.

- What data structure is it? Be specific.
- What is a better name for the Mystery variable **b**?
- For each of the operations **foo**, **bar** and **baz** on the data structure, state briefly (in one or two sentences) what they do and their asymptotic (worst-case) complexity. Also state the asymptotic *amortised* (worst-case) complexity if it is different.

data Mystery:

a: array of values

b: integer

function foo(m: Mystery):

i = 0

repeat:

g = i · 2 + 1

if not g < m.b:

return

if g + 1 < m.b **and** m.a[g+1] > m.a[g]:

g = g + 1

if m.a[g] ≤ m.a[i]:

return

tmp = m.a[i]

m.a[i] = m.a[g]

m.a[g] = tmp

i = g

function bar(m: Mystery) → value:

result = m.a[0]

m.a[0] = m.a[m.b-1]

m.a[m.b-1] = null

m.b = m.b - 1

if m.b < m.a.length / 4:

arr = **new array of length** m.b · 2

for i **in** 0 ... m.b-1:

arr[i] = m.a[i]

m.a = arr

foo(m)

return result

function baz(m: Mystery) → array of values:

arr = **new array of length** m.b

for i **in** 0 ... m.b-1:

arr[i] = bar(m)

return arr

Answers:

a) It's a max-heap.

b) "size" seems like a perfect name.

c) **foo**: It's a sink/sift down/bubble down operation on the root of the heap, $O(\log(n))$.

bar: A remove-max operation. $O(n)$, amortised $O(\log(n))$, because of resizing.

baz: Removes all values from the heap and returns them in descending order, $O(n \log(n))$.